

Lezione — Modulo S3: Web Security (OWASP Top Ten 2021)

Corso: Lab Sicurezza Informatica T **Materiale:** Web_security_6_marzo.pdf (45 pp.) + LAB_web_security_11_marzo.pdf (42 pp.) **Prerequisiti:** S1 (nmap, enumeration, surface d’attacco), S2 (autenticazione, sessioni) **Nota esame:** S3 è una delle 5 tipologie d’esame (**Web vulnerabilities**). Conta sia per il quiz teorico (40%) — con penalità per risposta sbagliata — sia per la prova pratica (60%). Le vulnerabilità OWASP compaiono sia come domande teoriche V/F sia come esercizi da eseguire su DVWA.

Come leggere questa lezione

Il materiale di S3 ruota attorno a una domanda sola: *cosa succede quando un’applicazione web si fida dell’input che riceve?* Quasi tutto ciò che OWASP classifica nasce da questa fiducia mal riposta. I ganci concreti sono i payload che incontrerai su DVWA — ' OR 'a'='a, <script>alert("XSS")</script>, ; ls — e gli header HTTP che il server può (o non può) impostare per difendersi. Leggi questa lezione capendo *perché* ogni tecnica funziona; la sequenza passo-passo su DVWA è nella guida-lab (/lab S3).

La visione d’insieme — threat model

Un’applicazione web è un sistema che riceve input dall’esterno (parametri URL, form, cookie, header, file XML) e li usa per costruire query, comandi, pagine HTML, chiamate ad API. L’**attaccante** sfrutta il fatto che l’applicazione non distingue tra *dato* e *istruzione*: se l’input dell’utente finisce dentro una query SQL senza essere sanificato, l’utente può scrivere SQL. Se finisce dentro una pagina HTML senza escaping, può scrivere JavaScript. Se finisce in una chiamata di sistema, può scrivere comandi bash.

La prospettiva del **difensore** è speculare: ogni punto dove l’input esterno influenza un interprete (database, shell, browser, parser XML) è una superficie d’attacco che va trattata con zero fiducia. Il principio è *validate input, escape output* — valida quello che entra, codifica tutto quello che esce verso un interprete. OWASP Top Ten 2021 cataloga le famiglie di errori più frequenti: non sono dieci cose separate, sono dieci facce dello stesso problema di trust mal gestito.

Il **lab** di S3 usa DVWA (Damn Vulnerable Web Application) su Docker, avviato con `pentestlab.sh`. DVWA espone intenzionalmente tutte le vulnerabilità OWASP, impostate al livello “low” (zero filtri). Questo ti permette di vedere la vulnerabilità nella sua forma più pura, capire il meccanismo, e poi alzare il livello per vedere come i filtri cambiano la superficie.

Pattern errore da errori_frequenti.md: prima di qualsiasi test su DVWA, costruisci la catena mentale — *che informazione voglio ottenere → quale vulnerabilità la espone → quale payload la sfrutta*. Perdere il filo (“cosa sto facendo?”) è il pattern “narrativa vs comandi” che hai già mostrato in S1. Ancòrati sempre all’obiettivo.

A1 — Broken Access Control: IDOR e Path Traversal

Insecure Direct Object Reference (IDOR) è il caso in cui una risorsa è referenziata direttamente da un identificatore (un numero, un nome file) che l'applicazione accetta senza verificare se l'utente che lo richiede ne abbia il diritto. L'attaccante non “buca” nulla in senso crittografico — cambia semplicemente `?id=42` in `?id=43` e legge i dati di qualcun altro. Il sistema di controllo accessi è assente o ignorato.

Il **File Disclosure / Path Traversal** è IDOR applicato a file: invece di un ID numerico, il parametro è un percorso. Se `?page=about.php` viene gestito includendo direttamente il file nominato, un attaccante può provare `?page=../../../../etc/passwd` e il server servirà il file delle password di sistema. La sequenza `../` sale di una directory — quanti `../` bastano dipende da dove si trova il web root, ma l'attaccante li accumula finché funziona.

LFI (Local File Inclusion) e **RFI (Remote File Inclusion)** sono la stessa vulnerabilità in due varianti: LFI include un file locale al server, RFI (disabilitato di default su DVWA) include un file remoto. Per RFI, l'attaccante serve un file PHP malevolo dalla propria macchina con `python3 -m http.server 8081` e punta il parametro `page=` verso quell'URL. Se il server PHP esegue il file incluso, l'attaccante ottiene esecuzione di codice remoto.

Dal lato difensore: non accettare mai percorsi di file come input utente. Se serve, usare una whitelist di pagine consentite e mai concatenare l'input direttamente in una chiamata al filesystem.

A2 — Cryptographic Failures

Questa categoria copre i dati sensibili (PII, credenziali, dati finanziari) che circolano o vengono conservati senza protezione crittografica adeguata. Non è solo “usare HTTP invece di HTTPS” — è anche trasmettere le password in chiaro nei log, usare algoritmi deboli (MD5 per le password, come vedrai su DVWA dove le password sono hash MD5 craccabili banalmente), o non proteggere il database at rest.

Il **meccanismo** che ti interessa per l'esame: un hash MD5 come `5f4dcc3b5aa765d61d8327deb882cf99` (che è l'hash di “password”) si cracca con una ricerca su qualunque rainbow table online. MD5 non è adatto alle password — manca di salt e di key stretching. Funzioni corrette sono `bcrypt`, `scrypt`, `Argon2`.

A3 — Injection: il cuore del lab

L'injection è la famiglia più ampia e quella su cui si concentra il lab pratico. Il principio è sempre lo stesso: dati non fidati vengono interpretati come codice da un interprete.

SQL Injection

L'applicazione DVWA costruisce la query così (dal sorgente PHP):

```
$getid = "SELECT first_name, last_name FROM users WHERE user_id = '$id'";
```

Il parametro `$id` viene inserito direttamente nella stringa SQL, senza prepared statement. Se digiti `1` ottieni il record con ID 1. Se digiti `' OR 'a'='a`, la query diventa:

```
SELECT First_Name,Last_Name FROM users WHERE ID='' OR 'a'='a';
```

La condizione 'a'='a' è sempre vera, quindi vengono restituiti tutti i record. Hai appena violato la logica di autorizzazione del database.

La tecnica **Union Based** porta questo più lontano. UNION in SQL permette di concatenare i risultati di due SELECT — ma richiede che le due SELECT abbiano lo stesso numero di colonne. La prima cosa da scoprire è quindi quante colonne ha la query originale. Si usa la tecnica dei NULL progressivi:

```
' union select NULL #          → errore (numero colonne sbagliato)
' union select NULL,NULL # → risultato! → la query ha 2 colonne
```

Una volta noto il numero di colonne, si può estrarre qualsiasi informazione dal database. I **metadati di sistema** si leggono con funzioni built-in di MySQL/MariaDB:

```
' union select NULL,@@version #    → versione del database
' union select NULL,@@hostname #   → hostname della macchina
' union select NULL,database() #   → nome del database corrente
```

La struttura del database si esplora attraverso `information_schema`, un database speciale di MySQL che contiene la mappa di tutti gli altri database, tabelle e colonne:

```
' union select null,schema_name from information_schema.schemata #
→ lista di tutti i database (dvwa, mysql, information_schema, ...)

' union select null,table_name from information_schema.tables where table_schema='dvwa' #
→ tabelle nel database dvwa: guestbook, users

' union select null,column_name from information_schema.columns where table_name='users' #
→ colonne della tabella users: user_id, first_name, last_name, user, password, avatar

' union select user,password from users #
→ estrae username e hash password di tutti gli utenti
```

Nota il # finale: in MySQL # è un commento che “tronca” il resto della query originale. Questo serve perché la query originale spesso ha ancora un ' di chiusura dopo \$id — senza il commento, la sintassi risultante sarebbe invalida.

Visione attaccante: con Union Based SQL injection hai accesso a tutto il database — non solo alla tabella originale, ma a qualsiasi schema presente nel DBMS. Le password sono hash MD5 su DVWA, craccabili offline. In un sistema reale questo è data breach completo.

Nota DBMS: la sintassi di `information_schema` è specifica di MySQL/MariaDB. PostgreSQL e SQLite hanno schemi equivalenti ma sintassi diversa. Su DVWA usi MySQL.

Command Injection

DVWA ha una pagina che accetta un IP e lo passa a `ping`. Il codice PHP usa `exec()` senza filtrare l'input. In bash, il carattere `;` permette di concatenare comandi: `cmd1; cmd2` esegue `cmd2` sempre. `&&` esegue `cmd2` solo se `cmd1` ha avuto successo (exit code 0).

Inserendo `127.0.0.1; ls` nella form del ping, ottieni sia l'output del ping sia il contenuto della directory corrente del server. La differenza tra `;` e `&&` nel contesto di command injection è pratica:

con `&&` se l'IP inserito è invalido e ping fallisce, il secondo comando non gira; con `;` gira sempre. Per l'attaccante, `;` è più affidabile.

Visione: command injection è RCE — Remote Code Execution. Con `;` `cat /etc/passwd;` `id;` `whoami` puoi esplorare il sistema con i privilegi del webserver (tipicamente `www-data`). Con `;` e i comandi giusti puoi risalire a una reverse shell.

XSS — Cross-Site Scripting

XSS sposta la prospettiva dal server al browser: l'iniezione non colpisce un interprete lato server (database, shell) ma l'interprete lato client (il motore JavaScript del browser della vittima).

Il payload classico è `<script>alert("XSS")</script>`. Se la pagina web non esegue escaping sull'output, questo tag viene interpretato come HTML dal browser e lo script viene eseguito. La prova di concetto con `alert()` è innocua, ma la stessa superficie permette payload molto più gravi: furto di cookie di sessione (`document.cookie`), keylogging, redirect verso siti di phishing, azioni per conto dell'utente.

Le **tre tipologie** hanno storie di vita diverse:

- **Reflected XSS:** il payload viaggia nell'URL, il server lo echo-a nella risposta, il browser lo esegue. Non viene salvato — il vettore è convincere la vittima a cliccare su un link costruito. Tipicamente: link in email di phishing.
- **Stored XSS:** il payload viene salvato nel database (commento in un forum, campo nome utente, post). Chiunque visiti la pagina che mostra quel dato esegue il codice malevolo. È il più pericoloso perché si propaga automaticamente a tutte le vittime che visitano la pagina infetta.
- **DOM-based XSS:** il payload non passa dal server — la pagina JavaScript legge un parametro dall'URL o dal DOM e lo inserisce nella pagina senza sanitizzazione, lato client. La risposta HTTP non contiene il codice malevolo (difficile da rilevare con WAF server-side).

Il **difensore** risponde con: encoding dell'output (convertire `<` in `<`, `>` in `>`; prima di inserire dati in HTML), Content Security Policy (header `Content-Security-Policy: default-src 'self'` che impedisce al browser di eseguire script da origini esterne), e header `X-XSS-Protection: 1; mode=block`.

A4 — Insecure Design

Vulnerabilità che non derivano da un bug implementativo ma dalla progettazione stessa del sistema. L'esempio del PDF: un sistema che permette password reset tramite domande di sicurezza prevedibili. Non è una SQLi, non è XSS — è una scelta architetturale sbagliata che non c'è patch che tenga.

A5 — Security Misconfiguration: l'errore di configurazione come vettore

A5 è la categoria più ampia: raccoglie tutti gli errori di configurazione a tutti i livelli dello stack (servizi di rete, server web, application server, database, framework, codice custom, VM, container,

storage). Non è un tipo di attacco specifico — è la condizione che rende possibili gli altri.

Credenziali di default

Il caso più ovvio: un'applicazione installata con le credenziali di default dell'installer. Su internet esistono database pubblici di credenziali di default per ogni dispositivo hardware, ogni DBMS, ogni immagine VM. Il lab usa proprio credenziali di default di DVWA (**admin/password**) come punto di partenza per il brute force — che poi scopre credenziali di default su un sistema esposto è esattamente questo scenario.

Cifrari TLS deboli

Scegliere versioni di protocollo o cifrari obsoleti espone la comunicazione anche se il sistema non ha bug applicativi. I protocolli da evitare: SSL, TLSv1.0, TLSv1.1. I cifrari da evitare: RC2, RC4, DES, Export, Client-Initiated Renegotiation. Da preferire: TLSv1.2+, cifrari a 128+ bit, Forward Secrecy (PFS — compromissione della chiave privata non svela le sessioni passate), chiavi asimmetriche 2048+ bit, SHA-2. Gli strumenti ssllabs.com e htbridge.com analizzano la configurazione TLS di un server e assegnano un rating (A = sicuro, F = vulnerabile).

HTTP Security Headers

Il server può istruire il browser a comportarsi in modo sicuro attraverso header HTTP di risposta. Sono la difesa in profondità lato client: anche se un attacco riesce a iniettare del contenuto, gli header possono limitarne l'esecuzione.

- **X-Frame-Options:** DENY o SAMEORIGIN — impedisce che la pagina sia caricata in un iframe da un sito esterno, mitigando clickjacking.
- **X-XSS-Protection: 1; mode=block:** attiva il filtro XSS del browser (legacy, ma ancora utile).
- **Content-Security-Policy: default-src 'self':** blocca il caricamento di risorse (script, immagini, font) da origini diverse dall'origine della pagina. Mitiga XSS stored e DOM-based.
- **X-Content-Type-Options: nosniff:** impedisce al browser di indovinare il MIME type della risposta — previene attacchi dove uno script viene mascherato da immagine.
- **Cache-Control: no-store + Pragma: no-cache + Expires: 0:** impedisce al browser di memorizzare risposte contenenti dati sensibili.
- **Strict-Transport-Security: max-age=<expire>** (HSTS): obbliga il browser a usare HTTPS per quel dominio, prevenendo HTTP stripping.
- **Public-Key-Pins:** lega la chiave pubblica di un dominio a un certificato specifico, prevenendo falsi certificati X.509.

SOP e CORS

Same Origin Policy (SOP): il browser impone che uno script caricato da `domain-a.com` non possa leggere risorse di `domain-b.com`. Senza SOP, un script iniettato dal dominio A potrebbe leggere i cookie di sessione del dominio B. È la difesa di base contro XSS cross-site.

CORS (Cross-Origin Resource Sharing) è il meccanismo per *rilassare* SOP in modo controllato quando due siti legittimi devono comunicare. Il browser aggiunge l'header `Origin: domain-a.com` alla richiesta cross-origin; il server di `domain-b.com` risponde con `Access-Control-Allow-Origin:`

domain-a.com (o * per tutti). Se l'ACAO non include l'origine richiedente, il browser blocca la risposta. CORS mal configurato (ACAO: * + credenziali) è una vulnerabilità, non una soluzione.

XXE — XML External Entities

XML permette di dichiarare “entità esterne” tramite DTD (Document Type Definition). Un parser XML “disattento” che non disabilita questa funzionalità può essere ingannato per leggere file dal filesystem del server. Il payload classico:

```
<?xml version="1.0" encoding="UTF-8"?>
<!DOCTYPE foo [ <!ENTITY xxe SYSTEM "file:///etc/passwd"> ]>
<stockCheck><productId>&xxe;</productId></stockCheck>
```

Il parser de-referenzia `&xxe;` come il contenuto di `/etc/passwd` e lo inserisce nella risposta — leak diretto del file. Con `SYSTEM` e un URI HTTP invece di `file://`, l'attacco diventa SSRF (il server fa richieste HTTP verso indirizzi interni che l'attaccante non può raggiungere direttamente). La variante DoS è la **Billion Laughs Attack**: entità annidate che si espandono esponenzialmente.

Mitigazioni: preferire JSON a XML dove possibile, usare parser aggiornati con external entity disabilitata per default, sanitizzare l'input XML consentendo solo elementi necessari.

A6 — Vulnerable and Outdated Components

Librerie, framework, OS con vulnerabilità note non patchate. Non è un errore di codice applicativo — è manutenzione mancata. I casi famosi sono istruttivi: **Heartbleed** (CVE-2014-0160, OpenSSL) permetteva di leggere memoria del server; **ShellShock** (CVE-2014-6271, Bash) permetteva RCE via variabili d'ambiente; **GHOST** (CVE-2015-0235, Linux glibc) era un buffer overflow nella risoluzione DNS. L'impatto potenziale è qualsiasi cosa — RCE, violazioni AAA — perché il componente vulnerabile può essere ovunque nello stack.

A7 — Identification and Authentication Failures

Il session management è il meccanismo che mantiene lo stato di autenticazione tra una request HTTP e l'altra (HTTP è stateless). Dopo il login, il server assegna un token (tipicamente un cookie) che il browser include automaticamente in ogni richiesta successiva.

Session Fixation

L'attaccante ottiene un token di sessione *prima* che la vittima si autentichi (ad esempio aprendo una sessione anonima). Convince poi la vittima a usare quel token — magari tramite phishing con link che include `?sessionId=123XYZ`. Quando la vittima si autentica, il server associa quella sessione autenticata al token noto all'attaccante, che ora può usarlo per impersonarla.

Un token di sessione è vulnerabile se: è prevedibile (sequenziale), è intercettabile (trasmesso in HTTP invece di HTTPS), non è legato al client specifico, non scade dopo inattività, non viene invalidato al logout.

CSRF — Cross-Site Request Forgery

Il browser include automaticamente i cookie di sessione in *ogni* richiesta verso un dominio, anche se la richiesta è generata da un altro sito. Un attaccante che controlla **attacker.com** può costruire una pagina con un form nascosto che punta a **mybank.com/transfer.jsp**, con i parametri pre-compilati (**recipient=attacker&amount=1000**). Quando la vittima, autenticata su mybank, visita attacker.com, il suo browser invia automaticamente la richiesta di bonifico con i cookie di mybank — il server non distingue la richiesta legittima da quella forgiata.

La **mitigazione** principale è il **CSRF token**: il server include in ogni form un segreto univoco, non falsificabile, non nell'URL (altrimenti è intercettabile nel Referer). Il form contiene `<input type="hidden" value="23a3af01b">`. Al submit, il server verifica che il token sia valido — un sito esterno non può conoscerlo e non può forgiare una richiesta valida.

Distinzione critica per l'esame (errori_frequenti.md): CSRF sfrutta l'*autenticazione* già esistente (il cookie di sessione) — non è una vulnerabilità di autorizzazione. La domanda “cosa fa CSRF?” ha risposta: sfrutta il fatto che il browser autentica automaticamente le richieste al dominio, non che i permessi siano configurati male.

A8 — Software and Data Integrity Failures

Vulnerabilità da insufficiente tutela dell'integrità del codice e dei dati dell'infrastruttura. Il caso più eclatante è **SolarWinds Orion** (2020): aggiornamenti legittimi del software di monitoraggio IT venivano firmati da SolarWinds ma contenevano codice malevolo iniettato durante il processo di build. 18.000 organizzazioni scaricarono l'aggiornamento; circa 100 furono compromesse seriamente, tra cui Microsoft, Intel, Cisco, agenzie governative USA.

La sottocategoria **Insecure Deserialization** (era A8 OWASP 2017, ora confluita qui) riguarda la serializzazione — il processo di convertire un oggetto in memoria in uno stream di byte per trasmetterlo (cookie, API, message broker) e poi ricostruirlo. Se il deserializzatore accetta stream non fidati senza verificarne l'integrità, un attaccante può costruire uno stream che, quando deserializzato, esegue codice arbitrario (Java, .NET) o eleva i propri privilegi (cookie con ruolo “admin”). Mitigazioni: non deserializzare dati non fidati, usare librerie con strict type checking, sandboxing.

A9 — Security Logging and Monitoring Failures

Non è una vulnerabilità che permette un attacco diretto, ma l'assenza di logging adeguato trasforma un attacco che avrebbe potuto essere rilevato e bloccato in uno che non viene scoperto per settimane. Errori comuni: non tracciare accessi falliti (il brute force passa inosservato), non dettagliare gli eventi loggati, non proteggere i log stessi da alterazione, non definire procedure di risposta agli incidenti. L'effetto: non si rileva il brute force, non si riesce a ricostruire la catena di compromissione, non si può stimare il danno subito.

A10 — Server-Side Request Forgery (SSRF) — new in 2021

Un'applicazione web riceve una URL dall'utente (es. “carica questa immagine”, “controlla questo feed RSS”) e la usa per fare una richiesta HTTP interna senza validarla adeguatamente. L'attaccante sostituisce l'URL con `http://169.254.169.254/` (metadata service di cloud AWS) o `http://192.168.1.1/` (router interno) — risorse non direttamente accessibili dall'esterno ma raggiungibili dal server. Risultato: enumerazione della rete interna, accesso a servizi interni protetti da firewall, potenzialmente RCE su sistemi non esposti. SSRF era già menzionato come effetto di XXE — la differenza è che qui è la logica applicativa stessa, non il parser XML, a fare la richiesta.

Connessioni

Con SysAdmin (3D — networking): molte vulnerabilità web dipendono da configurazioni di rete errate che SSRF e XXE sfruttano — un firewall mal configurato che espone i servizi interni amplifica enormemente il danno. La comprensione di `iptables` (S5) è direttamente connessa alla mitigazione di SSRF.

Con S1 (enumerazione): gobuster e nmap che hai usato in S1 sono i primi strumenti del lab S3. La fase di enumeration (trovare `/vulnerabilities/`, `/dvwa/`, directory nascoste) precede ogni exploit web. La catena S1→S3 è: scopro la superficie → identifico le vulnerabilità → le sfrutto.

Con S2 (autenticazione): A7 è direttamente il rovescio di S2. Quello che hai visto come “come funziona l'autenticazione corretta” in S2 corrisponde a “cosa succede quando l'autenticazione è rotta” in A7. Session fixation e CSRF sono gli attacchi a cui i meccanismi di S2 (token FIDO, sessioni robuste) rispondono.

Con S5 (firewall/iptables) e S10 (IDS Suricata): le difese lato rete contro i pattern di attacco web — rilevare port scanning, bloccare payload malevoli, loggare le anomalie — sono il contesto in cui A9 (Logging) diventa operativo.

Domande di autoverifica (stile quiz teorico)

All'esame c'è penalità per risposta sbagliata su domande V/F e a scelta multipla. Se non sei sicuro, non rispondere — il silenzio vale zero, la risposta sbagliata vale negativo.

1. Un attacco SQL Injection Union Based richiede di conoscere il numero di colonne della query originale prima di poter usare `UNION SELECT`. (*Vero/Falso*)

Risposta

Vero. `UNION` in SQL richiede che le due `SELECT` abbiano lo stesso numero di colonne e tipi compatibili. Si usa la tecnica dei `NULL` progressivi per scoprire il numero: `' union select NULL # (errore), ' union select NULL,NULL # (successo → 2 colonne)`.

2. In DVWA con difficulty “low”, inserire `' OR 'a'='a` nel campo User ID della sezione SQL Injection restituisce tutti i record della tabella users perché:

- a) Bypassa il sistema di autenticazione del web server
- b) La condizione 'a'='a' è sempre vera e la query restituisce tutti i record
- c) SQL interpreta OR come un operatore di shell e esegue il comando a
- d) Il parametro viene troncato e la query diventa una full table scan per default

Risposta

b. Il parametro `$id` viene inserito direttamente nella stringa SQL: `WHERE user_id = '$id'`. Con input `' OR 'a'='a'`, la query diventa `WHERE user_id = '' OR 'a'='a'` — la seconda condizione è sempre vera, quindi tutti i record passano il filtro.

3. CSRF (Cross-Site Request Forgery) sfrutta principalmente una debolezza nel sistema di *autorizzazione* dell'applicazione target. (*Vero/Falso*)

Risposta

Falso. CSRF sfrutta il meccanismo di *autenticazione* — il fatto che il browser include automaticamente i cookie di sessione in ogni richiesta verso quel dominio, anche se la richiesta proviene da un sito terzo. L'applicazione target non ha difetti di autorizzazione: il problema è che non distingue le richieste legittime dell'utente da quelle forgiate da un sito malevolo.

4. Un attaccante usa il payload XML:

```
<!DOCTYPE foo [ <!ENTITY xxe SYSTEM "file:///etc/passwd"> ]>
```

Cosa ottiene se l'attacco ha successo?

- a) Esecuzione di codice arbitrario sul server
- b) Il contenuto del file `/etc/passwd` del server nella risposta HTTP
- c) Un redirect verso `/etc/passwd` nel browser
- d) Un attacco DoS per espansione esponenziale delle entità

Risposta

b. XXE con `SYSTEM "file:///..."` istruisce il parser XML a de-referenziare l'entità come il contenuto del file locale. Il server restituisce il contenuto di `/etc/passwd` nella risposta (come mostrato nell'esempio del PDF). La risposta d) descrive la Billion Laughs Attack, che è una variante XXE diversa.

5. Quale delle seguenti affermazioni su DOM-based XSS è corretta?

- a) Il payload viene salvato nel database del server e colpisce tutti gli utenti che visitano la pagina

- b) La risposta HTTP del server contiene il codice malevolo
- c) Il payload viene eseguito nel browser della vittima senza che sia presente nella risposta HTTP del server
- d) È il tipo di XSS più facile da rilevare con un Web Application Firewall lato server

Risposta

c. DOM-based XSS agisce interamente lato client: JavaScript nella pagina legge un parametro dall'URL o da un'altra sorgente DOM e lo inserisce nel DOM senza sanitizzazione. La risposta HTTP non cambia — l'attacco non passa per il server, quindi i WAF server-side non lo vedono (risposta d è falsa).

Riepilogo

I dieci item OWASP sono manifestazioni dello stesso problema di fiducia mal gestita: A3 (Injection) è l'input che viene interpretato come codice; A1 è l'input che bypassa il controllo degli accessi; A5 è la configurazione che lascia porte aperte. Sulla VM DVWA le vulnerabilità sono visibili nella loro forma più pura (difficulty “low”) — l'obiettivo del lab non è “sbloccare il punteggio” ma capire il meccanismo di ognuna abbastanza da saper riconoscere e sfruttare la stessa classe di vulnerabilità in un contesto diverso all'esame.

Connessioni al grafo

- [[lezione_moduloS1_offensive_security_enumerazione]] — prerequisito: nmap, enumeration, superficie d'attacco
- [[lezione_moduloS2_autenticazione]] — prerequisito: sessioni, token, autenticazione vs autorizzazione
- [[guida_lab_moduloS1_enumerazione_nmap]] — tecnica di enumeration usata come primo step del lab S3